

Credit Card Fraud Detection Report

DEPLOYMENT TARGET
Streamlit Cloud

LOCAL ENGINE
WSL2 / Docker Desktop

PYTHON STACK
Python 3.13 / Streamlit

REPOSITORY
ruchitharuchi0575-ux

EXECUTIVE SUMMARY

This engineering report outlines the complete lifecycle of designing, optimizing, and deploying an end-to-end Machine Learning Credit Card Fraud Detection application. The system integrates real-time prediction capabilities via a Streamlit interface, local persistence using SQLite, and containerized deployment through Docker and Streamlit Community Cloud.

TIER 1: INFRASTRUCTURE STABILIZATION & LOCAL ENVIRONMENT

Initial execution on Windows 11 with WSL2 encountered hardware lockups and RAM exhaustion under heavy Docker builds. To stabilize the local development lifecycle, resource capping and engine optimizations were performed.

Key Challenge Resolved: System Freezes & Docker Daemon Disconnection

High RAM consumption during local container orchestration triggered host system lockups and corrupted the Windows Named Pipe (`npipe://./pipe/dockerDesktopLinuxEngine`). Capping the Virtual Machine resource allocation restored total host stability.

1. Resource Capping via `.wslconfig`

A global `.wslconfig` file was configured in the host system directory (`C:\Users\DELL\.wslconfig`) to constrain WSL2 memory and CPU consumption strictly to prevent out-of-memory kernel panics:

```
[wsl2]
memory=4GB
processors=2
swap=2GB
```

2. Execution Command Workflow for Container Management

To avoid high UI overhead and CodeLens-induced freezes in VS Code, container execution was transitioned strictly to background-detached CLI operations:

- **Apply Configuration:** `wsl --shutdown` (Forces WSL2 restart to apply 4GB ceiling).
- **Detached Execution:** `docker compose up -d` (Prevents interactive terminal locking).
- **Resource Monitoring:** `docker stats --no-stream` (Validates memory footprint).
- **Safe Teardown:** `docker compose down` or `docker stop credit_card_fraud_pipeline`.

TIER 2: APPLICATION ARCHITECTURE & DATA PIPELINE

The core analytical engine is structured inside `scripts/dashboard.py`. It provides a web dashboard designed to process real-time transaction inputs, evaluate fraud probability using a pre-trained model, and persist historical predictions.

Machine Learning Pipeline

Loads standard classification models (RandomForest / XGBoost) trained on normalized numerical features and PCA-transformed predictors V1-V28.

Data Storage (SQLite)

Stores every prediction record, user input parameters, confidence scores, and timestamped logs directly into a lightweight embedded database.

Core Script Structure: `scripts/dashboard.py`

The application follows a structured execution path from UI creation to model inference:

```
import streamlit as st
import pandas as pd
import numpy as np
import sqlite3
import joblib

# Database Initialization
def init_db():
    conn = sqlite3.connect("data/fraud_predictions.db")
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS predictions (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            amount REAL,
            fraud_probability REAL,
            is_fraud INTEGER,
            timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
        )
    """)
    conn.commit()
    conn.close()

st.title("Credit Card Fraud Detection Dashboard")
# Input sliders, feature scaling, model predict, and DB insert flow
```

Container Configuration Files

Configuration File	Purpose & Critical Settings	Modification Made
Dockerfile	Defines standard Linux image base (python:3.13-slim), installs requirements, and sets Streamlit endpoint.	Optimized layers to reduce build time and image size.
docker-compose.yml	Maps internal port 8501 to host port 8501 and manages container names.	Removed obsolete version: '3.8' attribute to fix modern engine warnings.

TIER 3: VERSION CONTROL & REMOTE SYNCHRONIZATION

Seamless cloud deployment requires a properly synchronized GitHub repository. During initial synchronization, placeholder remote URLs and branch name mismatches required targeted resolution.

Resolved Remote Synchronization Issue

Execution of `git push` initially failed with fatal: repository '`.../YOUR_USERNAME/...`' not found due to an unconfigured remote placeholder. Updating the remote origin directly resolved the link.

Step-by-Step Terminal Commands Executed

The following sequential commands established the clean Git state on GitHub:

```
# 1. Initialize local repository
git init

# 2. Stage all codebase files
git add .

# 3. Commit staged snapshot
git commit -m "Initial commit for Streamlit Cloud deployment"

# 4. Standardize default branch name to main
git branch -M main

# 5. Fix incorrect remote URL
git remote set-url origin https://github.com/ruchitharuchi0575-ux/creditcardFD.git

# 6. Authenticate and push codebase
git push -u origin main
```

Authentication & Credential Management

When prompted by the **Git Credential Manager** dialog box, authentication was completed by selecting `Sign in with your browser`. This bound VS Code directly to GitHub OAuth, permitting secure pushing of project assets without hardcoding access tokens.

Target Repository	Branch	Auth Provider	Sync Status
ruchitharuchi0575-ux/creditcardFD	main	GitHub OAuth (Browser)	SYNCHRONIZED

TIER 4: PRODUCTION DEPLOYMENT TO STREAMLIT CLOUD

With the repository successfully published on GitHub, production hosting was established via Streamlit Community Cloud, eliminating local resource usage completely.

1. Dependency Management File: requirements.txt

Streamlit Cloud requires an explicit manifest of Python dependencies located at the root of the repository to build the cloud container environment automatically:

```
streamlit
pandas
numpy
scikit-learn
joblib
```

2. Streamlit Cloud Deployment Settings

The cloud app creation workflow was configured with the following parameters:

- **Repository:** ruchitharuchi0575-ux/creditcardFD
- **Branch:** main
- **Main File Path:** scripts/dashboard.py
- **Custom URL:** https://creditcardfd-ux.streamlit.app

Deployment Troubleshooting Note

If Streamlit Cloud displays *"This branch does not exist"* or *"This file does not exist"*, it indicates stale browser caching from before the `git push`. A hard refresh (`Ctrl + F5`) forces the cloud deployer to re-fetch the latest GitHub HEAD commit.

COMPREHENSIVE SUMMARY OF SYSTEM CHANGES

Category	Initial State	Targeted Fix / Configuration	Final Outcome
System RAM	16GB Host Exhaustion	Capped WSL2 memory to 4GB in <code>.wslconfig</code>	System fast & stable
Docker Engine	<code>`npip`</code> daemon disconnect	Ran Docker Desktop as Administrator	Stable background service
Compose Spec	Obsolete version warning	Removed version: <code>'3.8'</code> line	Warning-free builds
Version Control	Placeholder Remote URL	Updated origin to explicit GitHub HTTPS path	Clean main tracking branch
Hosting	Local Docker Desktop	Pushed to Streamlit Community Cloud	Zero local RAM footprint